

Agent Observability & Eval Harness

A standalone, deep-dive reference for Build 4 — every phase, every file, every verification result, and every real gap found along the way, in full. Complements (does not replace) the living cheat sheet and the three per-phase flow diagrams.

STATUS

Complete — all 3 phases

COMMITTS

a10e9cd → 6b79732

CONTINUES

Build 3.5 (documentation RAG agent)

PRECEDES

Build 5 (live deployment with CI)

- [What & Why](#)
- [Phase 1](#)
- [Phase 2](#)
- [Phase 3](#)
- [Verification Log](#)
- [Known Limitations](#)
- [Environment State](#)

What Was Built, and Why

Every prior build added a new capability to the agent. Build 4 adds nothing the agent can *do* — it adds the ability to know, in detail, what the agent already did. The real gap: Build 1 Phase 5's "logging" is just a `print()` of each SQL statement with a timestamp — not durable, not queryable, and only ever covered `run_sql_query`, never `list_tables`, `describe_table`, `save_dataframe`, `search_summaries`, or `search_docs`.

Three phases: **(1)** structured tool-call tracing — every call logged with tool name, input, output, latency, turn number, timestamp; **(2)** an eval harness — a fixed set of test questions with expected criteria, run end-to-end against the live agent and reported pass/fail, genuinely different from the existing pytest suite (which tests tool functions in isolation, never agent behavior); **(3)** a metrics rollup over the log — tool-usage frequency, average turns per question, error rate. Deliberately not a Grafana-style dashboard, disproportionate at this project's scale.

Phase 1's foundational design decision: a JSON Lines file, not a new Postgres table.

`logs/tool_calls.jsonl` (gitignored) breaks from this project's "everything lives in Postgres" pattern on purpose — logs kept separate from operational data is the realistic production observability pattern (closer to what ships to Datadog/CloudWatch/Loki), and a genuinely distinct portfolio skill rather than "another Postgres table." Analyzing it later means a small Python script reading the file line-by-line, not a `SELECT` — exactly what Phase 2 and Phase 3 both do.

1 Structured Tool-Call Tracing

Goal: every tool call the agent makes gets logged — tool name, input, output, latency, turn number, timestamp — durable and queryable, covering all 5 tools, not just SQL.

```
logging_utils.py - log_tool_call()

def log_tool_call(tool_name, input_data, output_data, latency_ms, turn, question_id, error=None):
    os.makedirs(os.path.dirname(LOG_PATH), exist_ok=True)
    entry = {
        "timestamp": datetime.now().isoformat(),
        "question_id": question_id, # added in Phase 3 - see that section
        "turn": turn,
        "tool_name": tool_name,
        "input": input_data,
        "output": output_data if error is None else None,
        "error": error,
        "latency_ms": round(latency_ms, 2),
    }
    with open(LOG_PATH, "a") as f:
        f.write(json.dumps(entry, default=str) + "\n")
```

default=str in json.dumps is load-bearing, not decoration — run_sql_query results can contain datetime objects straight from Postgres, which the stdlib json module can't serialize natively.

```
agent.py - wiring (success + error paths)

start = time.perf_counter()
result = run_sql_query(block.input["sql"]) # (or whichever tool matched)
latency_ms = (time.perf_counter() - start) * 1000
log_tool_call(block.name, block.input, result, latency_ms, turn, question_id)
# ...

except Exception as e:
    latency_ms = (time.perf_counter() - start) * 1000
    log_tool_call(block.name, block.input, None, latency_ms, turn, question_id, error=str(e))
```

A real bug surfaced mid-edit — logging calls landed in the wrong branch entirely. An auto-applied edit inserted `log_tool_call(...)` inside the dispatch chain's `else: branch`, right before `raise ValueError` — wrong twice over. That branch only ever fires for an *unknown* tool name, never the real success path, and it references `result`, which is never assigned there — so it would itself throw `NameError` and mask the intended error entirely. Caught by reading the actual current file state rather than assuming a prior instruction had already

been applied correctly. Fixed by moving the calls to the real success path (after the dispatch chain) and the real error path (inside `except Exception as e`).

Readability follow-up, same day: console output was raw Python object reprs (`TextBlock(...)`, `ToolUseBlock(...)`), plus a duplicate text-print — the same response's text printed twice, once via a labeled loop and again via a leftover unlabeled one lower down. Fixed with labeled `[TEXT]/[TOOL CALL]/[TOOL RESULT]/[TOOL ERROR]` prints, `json.dumps(result, indent=2, default=str)` instead of a raw `print(result)`, and the redundant loop deleted outright.

Verified live against "Why does `agent_scratch` have two separate database roles?" — the model called `search_docs` twice (a broad first query, then a sharper reformulation once the first results proved too general) before synthesizing a correct answer. Every log entry checked out:

`timestamp/turn/tool_name/input/output/error: null/latency_ms` all present and correct.

A genuinely new observability finding, surfaced only because this logging now existed: the first `search_docs` call in a process takes ~2.4-3.0 seconds; the second call in the same run takes ~33ms — a ~90x difference, the one-time cost of lazily loading the embedding model into its module-level cache. Visible latency data that print-only logging never exposed — a concrete example of Phase 1 paying for itself before Phase 2 even started.

Sample – real logs/`tool_calls.jsonl` lines reproducing this same finding (output truncated for length)

```
{
  "timestamp": "2026-07-23T13:27:28.628848",
  "question_id": "1c2ed444016c",
  "turn": 1,
  "tool_name": "search_docs",
  "input": {"query_text": "agent_scratch database roles..."},
  "output": [...],
  "error": null,
  "latency_ms": 2980.59
}
{
  "timestamp": "2026-07-23T13:27:29.979725",
  "question_id": "1c2ed444016c",
  "turn": 2,
  "tool_name": "search_docs",
  "input": {"query_text": "appdb_reader appdb_agent_writer..."},
  "output": [...],
  "error": null,
  "latency_ms": 109.97
}
```

Same `question_id`, turn 1 vs. turn 2: 2980.59ms → 109.97ms (~27x here) - the exact multiplier varies run to run, but the lazy-load pattern reproduces consistently on every fresh process.

2 The Eval Harness

Goal: a fixed set of test questions with expected criteria (which tool should fire, what the answer should contain), run end-to-end against the live agent — genuinely different from the pytest suite, which never exercises agent behavior at all.

```
agent.py - refactored into a callable ask_agent()

def ask_agent(user_question, max_tool_turns=MAX_TOOL_TURNS):
    question_id = uuid.uuid4().hex[:12] # added later, in Phase 3
    messages = [{"role": "user", "content": user_question}]
    turn = 0
    answer = ""
    while turn < max_tool_turns:
        # ... unchanged tool-use loop ...
        if response.stop_reason != "tool_use":
            return {"answer": answer, "error": None}
    return {"answer": answer, "error": "max_turns_reached"}

if __name__ == "__main__":
    # old hardcoded question, preserved - still runnable directly for manual testing
    ask_agent("Why does agent_scratch have two separate database roles?")
```

Every break/loop-fallthrough became a return in the same spot — behaviorally identical, since returning exits the function exactly like break exited the loop. `answer = block.text` on every text block means the returned answer is the *last* text block seen, not an intermediate "I'll search for..." message. The `if __name__ == "__main__":` guard preserves the old CLI behavior exactly, so `agent.py` stayed both directly runnable and safely importable. Verified two ways: `py_compile`, and a full live run confirming identical behavior to the pre-refactor version.

```
eval_cases.py - the case schema

{
    "question": "...",
    "expected_tool": "...",          # must appear among tools the agent actually called
    "expected_keywords": [...],     # all must appear in the final answer text (case-insensitive)
    "cleanup_table": "...",        # optional - dropped after the case runs
}
```

7 cases, covering all 5 tools plus the enumeration-vs-thematic edge case Build 3.5 surfaced. The `save_dataframe` case uses a distinct eval-only table name (`eval_domain_avg_duration`, not the real

domain_avg_duration from Build 2.5) plus a cleanup_table field, mirroring test_agent_scratch_boundary.py's create-verify-drop pattern so the suite stays rerunnable.

run_eval.py - the core loop

```
for case in EVAL_CASES:
    start_line = _log_line_count()
    response = ask_agent(case["question"])
    new_entries = _new_log_entries(start_line) # reuses Phase 1's log as the source of truth

    tools_called = sorted({entry["tool_name"] for entry in new_entries})
    tool_ok = case["expected_tool"] in tools_called

    answer_lower = response["answer"].lower().replace(", ", "")
    keywords_ok = all(kw.lower() in answer_lower for kw in case["expected_keywords"])
    passed = tool_ok and keywords_ok and response["error"] is None
```

Design choice: reuse Phase 1's log instead of having ask_agent() return tool-call metadata directly. Avoids duplicating tracking logic that already exists, and is a direct, concrete payoff of Phase 1's investment. Cleanup uses pycpg.sql.Identifier, matching save_dataframe's safe-identifier convention — not defending against a real injection risk here (the name only ever comes from trusted eval_cases.py), but no reason to break the project's own established pattern.

First live run found a real turn-budget bug, not just a bad test. The "list all SQL migration files" case hit max_turns_reached before ever answering. Root cause, traced turn by turn: describe_table('docs.chunks') silently returned [], because describe_table matches on bare table name only across all schemas at once — a schema-qualified name never matches, and nothing said so. That sent the agent down several dead-end ILIKE '%migration%' / ILIKE '%.sql' queries (the docs never use the literal word "migration"; the un-anchored %.sql pattern only matches text *ending* in .sql) before it ever found the right query — burning the entire 10-turn budget along the way.

Fix, applied and re-verified live: MAX_TOOL_TURNS raised 10→15; describe_table's tool description now states plainly it's a bare-name match; SYSTEM_PROMPT gained a concrete steer — query chunk_text WHERE source_file = 'sql-test-01-cheatsheet.html' directly for "list all X" questions, since that file already enumerates everything. Re-run: the same question now converges in a single run_sql_query call with a complete, correct answer.

Keyword-robustness lesson, found twice in the same build. After the turn-budget fix, the migration case's expected_keywords: ["docs.chunks"] still failed — not an agent bug, a bad eval guess (an internal table

name a correct prose answer would never say back to the user). Separately, a previously-passing role-boundary case failed on a later re-run whose answer was fully correct but phrased differently — free-text paraphrase varies run to run even when the underlying reasoning is right. **Both fixed the same way:** swap paraphrase-dependent keywords for stable identifiers the answer is structurally required to contain — ["001_schema.sql", "009_add_summary_embedding_hnsw_index.sql"] (real filenames, first/last in sequence, catches truncation too) and ["appdb_reader", "appdb_agent_writer"] (the actual role names) respectively.

Round 2 — eval_cases_round2.py, 5 new cases, each verified against ground truth queried directly from the DB before being written (a stronger, independent check than round 1 mostly used, since it doesn't risk the eval and the agent being wrong the same way): department aggregation (ILLU, 128 named-department allocations, NULL departments excluded), the previously-untested customers table (4 rows), rediscovering a scratch table saved back in Build 2.5, an exact-filter discrimination case (renewal_count = 0 → 1243 rows, testing that the agent picks run_sql_query over search_summaries for a structured question), and a graceful-no-match case for search_summaries (equipment that doesn't exist in the dataset — confirms the "no sufficiently relevant results" message survives into the final answer without the model hallucinating a match).

run_eval.py generalized to accept multiple case files: sys.argv[1] (default "eval_cases") selects which module's EVAL_CASES to run — python run_eval.py eval_cases_round2 runs the new file with no runner edits. Also hardened against numeric-formatting variance: answer_lower strips commas before the keyword check, confirmed necessary live when a round-2 answer used "1,243" against an expected keyword of "1243".

Result: 7/7 and 5/5 passing — 12/12 across both rounds, after the turn-budget and keyword-robustness fixes above.

Sample – real EVAL REPORT output, round 1 (3 of 7 cases shown; full output in logs/eval_round1_output.log)

```
[PASS] Why does agent_scratch have two separate database roles?
tools called: ['search_docs'] (tool_ok=True)
keywords_ok: True
answer: Based on the project documentation, **agent_scratch has two separate
database roles for permission isolation and scope control**: 1. `appdb_agent_writer`...
```

```
[PASS] List all the SQL migration files used in this project.
tools called: ['run_sql_query'] (tool_ok=True)
keywords_ok: True
answer: Based on the project's cheat sheet, here are all the SQL migration files
used in this project: ## PostgreSQL Migrations 1. **001_schema.sql** - Original...
```

```
[PASS] What's the total revenue from all orders?
tools called: ['describe_table', 'list_tables', 'run_sql_query'] (tool_ok=True)
```

keywords_ok: True
answer: The **total revenue from all orders is \$438.48**.

=====

7/7 passed

=====

Sample - real EVAL REPORT output, round 2 (3 of 5 cases shown; full output in logs/eval_round2_output.log)

[PASS] Which named patron department has the most recorded allocations?
tools called: ['describe_table', 'list_tables', 'run_sql_query'] (tool_ok=True)
keywords_ok: True
answer: **The named patron department with the most recorded allocations is ILLU with 128 allocations.**

[PASS] How many allocations were never renewed (a renewal_count of zero)?
tools called: ['describe_table', 'list_tables', 'run_sql_query'] (tool_ok=True)
keywords_ok: True
answer: **Answer: There are 1,243 allocations that were never renewed (renewal_count = 0).**

[PASS] Is there any scuba diving equipment among the allocations?
tools called: ['search_summaries'] (tool_ok=True)
keywords_ok: True
answer: Based on the semantic search results, there does not appear to be any scuba diving equipment among the allocations in the database...

=====

5/5 passed

=====

3 The Metrics Rollup

Goal: a plain-text report over the tool-call log — tool-usage frequency, average turns per question, error rate. Deliberately not a dashboard.

A real design fork surfaced before any rollup code was written. Two of the three metrics are simple counts. The third, "average turns per question," is not — `logs/tool_calls.jsonl` had no session/question boundary field, only a turn number that restarts at 1 per `ask_agent()` call. Two options: **(1)** infer question boundaries by watching for turn resetting to 1 — zero schema change, but silently wrong the moment two conversations log concurrently and their lines interleave (two different conversations both starting at turn 1 around the same moment are indistinguishable without an explicit ID); **(2)** add a real `question_id` field now.

Decision: option 2, explicitly because Build 5 and Build 6 are real, still-planned future states of this project, not hypotheticals. Build 5 (a live deployed API) and Build 6 (a multi-user Streamlit front end) will eventually make concurrent `ask_agent()` calls possible — this was judged the right point to build for that, rather than retrofitting once real concurrent traffic exists.

`agent.py` — `question_id` generation

```
def ask_agent(user_question, max_tool_turns=MAX_TOOL_TURNS):
    # Random, not a counter - a shared counter would itself need synchronization once
    # Build 5/6 make concurrent ask_agent() calls possible, defeating the point of this ID.
    question_id = uuid.uuid4().hex[:12]
```

Threaded through both `log_tool_call` call sites (success and error paths). The pre-existing `logs/tool_calls.jsonl` (hundreds of lines from Phase 1/2 testing, no `question_id` at all) was archived locally as `tool_calls.pre-question-id.jsonl` rather than taught to the rollup as a second schema to branch on — the historical latency-lazy-load finding from Phase 1 is already captured in this brief, so nothing real was lost.

Verified live: two sequential `ask_agent()` calls produced distinct `question_ids`, with both turns of the first call correctly sharing one ID. The already-committed Phase 3 diagram's Step 2 was then amended (not left silently stale) to name this schema extension as a prerequisite of writing the rollup script — re-verified via both `--screenshot` and `--print-to-pdf + pdftoppm` before re-committing.

`metrics_rollup.py`

```
def main():
    entries = load_entries()
    tool_counts = Counter(e["tool_name"] for e in entries)
```

```

# turn is 1-indexed and increments once per round-trip within one question, so the
# highest turn value seen for a question_id IS the total turns that question took.
turns_per_question = {}
for e in entries:
    qid = e["question_id"]
    turns_per_question[qid] = max(turns_per_question.get(qid, 0), e["turn"])
avg_turns = sum(turns_per_question.values()) / len(turns_per_question)

error_count = sum(1 for e in entries if e["error"] is not None)
error_rate = error_count / len(entries)
# ... prints the three sections as a plain-text report ...

```

Verified live, and it surfaced something worth explaining: 52 tool calls across 21 questions, not the ~14 expected. Investigation traced it to two independent, legitimate invocations of the 7-case round — one run manually by the user, one run separately via this session — both appending to the same shared log, plus the 5-case round once and the 2 earlier verification questions ($7+7+5+2 = 21$). Not a bug: the report is correctly reflecting exactly what happened. It's also a quiet confirmation that `question_id` holds up even across genuinely separate process invocations sharing one log file — every question's turns grouped and averaged cleanly, with zero cross-contamination between the two runs. (Sequential, not concurrent, so this doesn't fully exercise the Build 5/6 scenario the field was added for — but it's a good sign nothing about the grouping is fragile.)

Sample – full real console output from a single `metrics_rollup.py` run (post follow-up: includes Trend and Log rotation, not part of Phase 3's original scope)

```

=====
METRICS REPORT
=====

Total tool calls logged: 52
Total questions asked:   21

--- Tool-usage frequency ---
run_sql_query: 16
list_tables: 14
describe_table: 13
search_docs: 4
search_summaries: 3
save_dataframe: 2

--- Average turns per question ---
2.29

--- Error rate ---
1/52 calls errored (1.9%)

```

--- Trend vs previous run ---
Previous run: 2026-07-23T22:21:11.629534+00:00
Total calls: 52 -> 52 (+0)
Total questions: 21 -> 21 (+0)
Avg turns/question: 2.29 -> 2.29 (+0.00)
Error rate: 1.9% -> 1.9% (+0.0%)

--- Log rotation ---
52/1000 lines - no rotation needed yet.

=====

METRIC	RESULT (52 CALLS / 21 QUESTIONS)
Tool-usage frequency	run_sql_query: 16, list_tables: 14, describe_table: 13, search_docs: 4, search_summaries: 3, save_dataframe: 2
Average turns per question	2.29
Error rate	1/52 (1.9%)

Verification Log

What was actually exercised against the running system, not just read for plausibility — same standard as every build in this project.

Phase 1 live run

Two `search_docs` calls (self-corrected reformulation), every log field present and correct, ~90x lazy-load latency gap discovered directly from the new data.

Phase 2, round 1

7/7 passing after the turn-budget and keyword-robustness fixes; migration-files case confirmed converging in a single tool call with a complete, correct answer.

Phase 2, round 2

5/5 passing on first run, every keyword pulled from a direct DB query, not copied from the agent's own output.

Phase 3, question_id grouping

Two sequential `ask_agent()` calls verified to produce distinct IDs with correct per-call turn grouping; later, 21 real questions across two independent invocations grouped with zero cross-contamination.

Phase 3 diagram amendment

Re-verified via both `--screenshot` and `--print-to-pdf + pdftoppm` after Step 2's text changed — no overflow in either render.

Full pytest suite, post project-wide comment sweep

19/19 still passing — confirms the comment-only pass across every project file (tools.py, the eval harness, Build 2/3/3.5 scripts, all 4 test files) changed no behavior.

Sample – real `pytest -v` output (5 of 19 tests shown)

```
test_agent_scratch_boundary.py::test_writer_has_no_elevated_privileges PASSED [ 5%]
test_agent_scratch_boundary.py::test_scratch_table_cap_blocks_new_tables_but_not_existing_ones
PASSED [ 26%]
test_data_quality.py::test_clean_department_has_real_nulls PASSED [ 42%]
test_docs_search.py::test_threshold_regression_guard PASSED [ 68%]
test_semantic_search.py::test_hnsw_index_exists_on_summary_embedding PASSED [ 78%]
```

```
===== 19 passed in 11.18s =====
```

Known Limitations & Open Items

Resolved, 2026-07-23 — `metrics_rollup.py` now persists a tracked time series, not just a one-shot snapshot. A new migration (`011_metrics_history_schema.sql`) adds `observability.metrics_history` — a dedicated schema, deliberately separate from `agent_scratch`, since this is written by a human-run maintenance script connecting as the admin role (same pattern as `ingest.py/embed_summaries.py`), not the agent itself at runtime. Each run inserts one row (`total_calls`, `total_questions`, `avg_turns_per_question`, `error_count`, `error_rate`, `tool_usage` as JSONB) and, critically, also queries the previous row and prints an explicit "Trend vs previous run" delta — storage alone wouldn't have answered "how has error rate trended since last week" without something surfacing the comparison. Verified live across 4 real runs, including catching and fixing a real cosmetic bug along the way: a `Decimal`→`float` precision mismatch produced a tiny epsilon that displayed as `-0.00` for a truly unchanged value, misleadingly reading as a regression — fixed by rounding to display precision *before* the zero-check, not after (rounding after checking never catches an epsilon that isn't exactly zero).

Sample - `observability.metrics_history`, real rows from this session's verification (psql)

id	run_at	total_calls	avg_turns_per_question	error_rate
1	2026-07-23 22:19:59.07071+00	52	2.29	0.0192
2	2026-07-23 22:20:08.781081+00	52	2.29	0.0192
3	2026-07-23 22:20:45.281438+00	52	2.29	0.0192
4	2026-07-23 22:21:11.629534+00	52	2.29	0.0192

(4 rows)

Sample - `metrics_rollup.py`'s "Trend vs previous run" section, real output

```
--- Trend vs previous run ---
Previous run: 2026-07-23T22:21:11.629534+00:00
Total calls:      52 -> 52 (+0)
Total questions:  21 -> 21 (+0)
Avg turns/question: 2.29 -> 2.29 (+0.00)
Error rate:       1.9% -> 1.9% (+0.0%)
```

Resolved, 2026-07-23 — `metrics_rollup.py` now rotates the log automatically instead of letting it grow forever. A `LOG_ROTATION_THRESHOLD_LINES` constant (1000 - an arbitrary safety cap, not a real capacity limit, same spirit as `MAX_SCRATCH_TABLES`) is checked after this run's rollup is already persisted to `observability.metrics_history` - rotation only ever happens once the data it would discard is already durably captured elsewhere, so nothing analytically valuable is lost. Past the threshold, `logs/tool_calls.jsonl` is renamed to a timestamped archive (`logs/tool_calls.<YYYYMMDD-HHMMSS>.jsonl`); `logging_utils.py` already creates the file fresh on the next call with no changes needed there. Verified live with a temporarily-

lowered threshold (monkeypatch-style override, same pattern as the MAX_SCRATCH_TABLES boundary test) rather than generating 1000 real log lines to trigger it - confirmed the rotation fires, archives correctly, and the real log/history data was restored afterward rather than left in an artificially-rotated state.

Sample - real output, both outcomes observed this session

--- Log rotation ---

52/1000 lines - no rotation needed yet.

--- Log rotation --- (LOG_ROTATION_THRESHOLD_LINES temporarily overridden to 10, to exercise the path)

52 lines >= 10 threshold - archived to logs/tool_calls.20260723-153329.jsonl

logs/tool_calls.jsonl will start fresh on the next tool call.

The turn-budget/enumeration fix depends on the model actually following the system-prompt steer each time — no hard guarantee. Same category of limitation Build 3.5 already flagged for its own enumeration fix; worth watching if this pattern gets exercised more heavily later.

question_id's concurrency-safety is currently unexercised, not unverified-by-design. Every real test this build ran was sequential (one process, one invocation at a time, or two separate sequential invocations sharing a log). The design is sound for genuine concurrency (a fresh random ID per call, no shared mutable state), but nothing in this build actually ran two ask_agent() calls at the same literal moment — that real test only becomes possible once Build 5 or Build 6 exists.

Environment State

COMPONENT	DETAIL
New Python	logging_utils.py, eval_cases.py, eval_cases_round2.py, run_eval.py, metrics_rollup.py
Changed Python	agent.py (ask_agent() refactor, question_id, MAX_TOOL_TURNS 10→15, describe_table/docs.chunks guidance), tools.py (comment-only)
New log directory	logs/ (gitignored) – tool_calls.jsonl (live), tool_calls.pre-question-id.jsonl (archived)
New dependency	none (uuid is stdlib)
Diagram folder	PROJECT_DIAGRAMS/BUILD_4_FLOW/ – phase1/phase2/phase3 flow diagrams, this brief
Role used	no new DB roles this build – appdb_reader / appdb_agent_writer, unchanged from Build 2.5
New SQL (follow-up, 2026-07-23)	011_metrics_history_schema.sql – observability.metrics_history, addressing the one-shot-snapshot limitation above
Eval suite total	12 cases across 2 rounds, 12/12 passing
Project test suite	19/19 passing (unchanged from Build 3.5 – this build's tests are the eval harness, deliberately separate from pytest)

Build 4 complete. Next: Build 5 — live deployment with CI (GitHub Actions). Ongoing cross-build command reference lives in `sql-test-01-cheatsheet.html`.